identifiers that look like relational database primary keys. You might use code like this for a proof of concept application or as a test double (a replacement for a relational database) for unit testing. Or indeed, you could use code like this as part of a demonstration application for learning purposes if you ever write a book! The only slightly odd thing here is that the `GetAllAsync` method returns a `Task<List<City>>` rather than just a `List<City>`. This is because you will be migrating to using a database in the next chapter, so you want to emulate database calls, which are typically executed asynchronously within an ASP.NET Core application. I discuss this in more detail in the next chapter.

Now that you have centralized the creation of cities, you can use your new component in the PropertyManager \Create.cshtml.cs file. Open that, and change the existing handler methods and `GetCityOptions` method, leaving the various page properties as they are. You will also need to add a `using` directive for `CityBreaks.Services`.

**Listing 7.2 Revised Create Property page using the new `SimpleCityService`**

```
public async Task OnGetAsync()                                          ❶
{
    Cities = await GetCityOptions();                                    ❶
}

public async Task OnPostAsync()                                         ❶
{
    Cities = await GetCityOptions();                                    ❶
    if (ModelState.IsValid)
    {
        var city = Cities.First(o => o.Value == SelectedCity.ToString());
        Message = $"You selected {city.Text} with value of {SelectedCity}";
    }
}

private async Task<SelectList> GetCityOptions()                         ❶
{
    var service = new SimpleCityService();                              ❷
```